UTS

PEMBELAJARAN MESIN



NAMA : Fajar Febriano Admosa

NIM : 202431164

KELAS : F

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC :

ASISTEN : 1. Jonathan Juan Setyanto

2. Muhammad Caisar Gemilang Pratama

3.

4.

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

BAB I PENDAHULUAN ..... **Error! Bookmark not defined.**
1.1 Rumusan Masalah ..... **Error! Bookmark not defined.**
1.2 Tujuan Masalah ..... **Error! Bookmark not defined.**
1.3 Manfaat Masalah ..... **Error! Bookmark not defined.**
BAB II LANDASAN TEORI ..... **Error! Bookmark not defined.**
2.1 Klasifikasi Biner pada Data Bank Marketing 4
2.2 Preprocessing dan Ketidakseimbangan Kelas 4
2.3 Regresi Linear 4
2.4 Decision Tree 5
2.5 Metrik Evaluasi 5
2.6 Penelitian Terkait 6
BAB III HASIL ..... **Error! Bookmark not defined.**
3.1 Lingkungan Pengerjaan dan Dataset 7
3.2 Penjelasan Kode dan Output Setiap Cell 8
3.3 Hasil Evaluasi Model 18
3.4 Analisis dan Pembahasan 19
BAB IV PENUTUP ..... **Error! Bookmark not defined.**
4.1 Kesimpulan ..... **Error! Bookmark not defined.**
4.2 Saran ..... **Error! Bookmark not defined.**
DAFTAR PUSTAKA ..... **Error! Bookmark not defined.**

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Bank menjalankan kampanye pemasaran melalui telepon untuk menawarkan produk deposito berjangka kepada nasabah. Tidak semua nasabah yang dihubungi bersedia berlangganan, sehingga biaya dan waktu kampanye berpotensi terbuang. Berdasarkan hasil UTS praktikum Pembelajaran Mesin dengan dataset Bank Customers, rumusan masalahnya adalah sebagai berikut.

1. Bagaimana karakteristik dataset Bank Customers berdasarkan hasil eksplorasi data (EDA), dan tahapan preprocessing apa yang diperlukan agar data siap dilatih?
2. Bagaimana menerapkan algoritma Regresi Linear dan Decision Tree untuk memprediksi apakah seorang nasabah akan berlangganan deposito berjangka?
3. Berapa akurasi yang diperoleh masing-masing model, dan model mana yang lebih baik pada dataset ini?

1.2 Tujuan Masalah

1. Memahami dataset Bank Customers (45.211 baris dan 17 kolom) melalui proses EDA dan preprocessing: tidak ada missing value maupun data duplikat, namun kelas target sangat tidak seimbang (88,3% "no" dan 11,7% "yes").
2. Melatih model Regresi Linear dan Decision Tree serta menghitung akurasinya pada data uji, yaitu 89,84% untuk Regresi Linear dan 89,90% untuk Decision Tree hasil tuning.
3. Menganalisis hasil pengolahan data berdasarkan teori dan data pendukung, termasuk perbandingan kedua model dan keterbatasannya.

1.3 Manfaat Masalah

1. Bagi mahasiswa: memahami alur lengkap pembelajaran mesin, mulai dari memuat data, EDA, preprocessing, pelatihan model, hingga evaluasi, serta mampu menjelaskan perbedaan karakteristik Regresi Linear dan Decision Tree.
2. Bagi bank: model dapat menjadi dasar untuk menyaring nasabah yang berpeluang berlangganan sehingga kampanye telepon lebih tepat sasaran dan lebih hemat biaya.
3. Bagi pengembangan selanjutnya: hasil ini menjadi acuan (baseline) untuk meningkatkan kinerja, misalnya dengan penanganan ketidakseimbangan kelas atau algoritma lain.

BAB II

LANDASAN TEORI

2.1 Klasifikasi Biner pada Data Bank Marketing

Prediksi apakah nasabah akan berlangganan deposito berjangka termasuk masalah klasifikasi biner dalam pembelajaran terbimbing, dengan keluaran "yes" atau "no". Dataset bank marketing dari UCI Machine Learning Repository berasal dari kampanye telepon sebuah bank di Portugal dan banyak dipakai sebagai studi kasus prediksi langganan deposito (Hayder et al., 2023). Versi dataset yang berisi 45.211 instans dan 17 atribut tanpa missing value juga dipakai oleh Aldelemy dan Abd-Alhameed (2023). Dataset ini juga digunakan untuk membandingkan berbagai klasifikator, baik yang bersifat parametrik maupun nonparametrik (Hayder et al., 2023).

2.2 Preprocessing dan Ketidakseimbangan Kelas

Kualitas data sangat memengaruhi hasil pemodelan. Tahap preprocessing umumnya mencakup pengecekan nilai hilang dan nilai tidak diketahui, pengubahan fitur kategorikal menjadi numerik, serta normalisasi data (Aldelemy & Abd-Alhameed, 2023; Hayder et al., 2023). Pembagian data latih dan uji perlu dilakukan secara terstratifikasi agar proporsi kelas pada kedua bagian tetap sama dengan data asli (Aldelemy & Abd-Alhameed, 2023).

Dataset bank marketing bersifat tidak seimbang: nasabah yang menolak jauh lebih banyak daripada yang menerima. Pada kondisi ini akurasi menjadi kurang informatif karena model yang cenderung memilih kelas mayoritas tetap tampak akurat (Hayder et al., 2023). Untuk mengatasinya, penelitian terdahulu menggunakan teknik penyeimbangan seperti random under-sampling dan SMOTE (Hayder et al., 2023) serta resampling yang dikombinasikan dengan seleksi fitur (Safarkhani & Moro, 2021). Karena itu evaluasi sebaiknya melengkapi akurasi dengan precision, recall, dan F1-score.

2.3 Regresi Linear

Regresi linear memodelkan hubungan linear antara sejumlah fitur dan target numerik. Untuk n fitur, model dituliskan sebagai y = b0 + b1x1 + b2x2 + ... + bnxn, dengan b0 sebagai intersep dan bi sebagai koefisien tiap fitur. Koefisien diestimasi dengan metode Ordinary Least Squares (OLS) yang meminimalkan jumlah kuadrat galat, SSE = sum(yi - y_hat_i)^2. Tanda koefisien menunjukkan arah pengaruh fitur, sedangkan nilai mutlaknya menunjukkan besar pengaruh, seperti dijelaskan untuk model berbasis kombinasi linear oleh Aldelemy dan Abd-Alhameed (2023).

Regresi linear menghasilkan nilai kontinu, sehingga untuk klasifikasi biner target dikodekan 0/1 dan hasil prediksi diubah menjadi kelas dengan ambang (threshold) 0,5: prediksi 0,5 atau lebih menjadi kelas 1 ("yes"). Pendekatan ini disebut *linear probability model*. Berbeda dari regresi logistik yang memodelkan keluaran biner melalui fungsi sigmoid (Aldelemy & Abd-Alhameed, 2023), keluaran regresi linear tidak dibatasi pada rentang 0 sampai 1 sehingga kurang ideal untuk target kategorikal. Kinerja regresi dinilai dengan R2, MAE, MSE, dan RMSE, sedangkan kinerja klasifikasinya dengan akurasi setelah thresholding.

2.4 Decision Tree

Decision Tree adalah model berbentuk pohon dengan simpul yang menguji suatu fitur, cabang yang mewakili hasil pengujian, dan daun yang menyatakan kelas. Pohon dibangun secara top-down melalui pemisahan data berulang (*recursive partitioning*) yang bersifat *greedy*, berhenti ketika data pada suatu simpul sudah seragam atau pemisahan tidak lagi menambah manfaat (Aldelemy & Abd-Alhameed, 2023). Pemilihan fitur dan nilai pemisah didasarkan pada ukuran impurity, yaitu *Gini = 1 - sum(pi^2)* atau *entropy = -sum(pi log2 pi)*; pemisahan terbaik adalah yang paling menurunkan impurity (information gain).

Decision Tree dikenal sebagai model *white-box* yang mudah dipahami karena alur keputusannya dapat ditelusuri, tetapi pohon yang dibiarkan tumbuh penuh rentan overfitting (Aldelemy & Abd-Alhameed, 2023). Overfitting dikendalikan dengan membatasi hiperparameter seperti *max_depth* dan *min_samples_leaf*, yang juga disetel pada penelitian terdahulu (Aldelemy & Abd-Alhameed, 2023). Pada data bank marketing, Decision Tree dilaporkan memperoleh akurasi sekitar 91 sampai 92% (Hayder et al., 2023) dan J48, varian Decision Tree, mencapai 94,39% setelah dikombinasikan dengan resampling dan seleksi fitur (Safarkhani & Moro, 2021).

2.5 Metrik Evaluasi

Evaluasi klasifikasi memakai *confusion matrix* yang terdiri dari *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). Dari matriks ini diturunkan metrik berikut (Aldelemy & Abd-Alhameed, 2023).

Metrik	Rumus	Makna
Akurasi	(TP + TN) / (TP + TN + FP + FN)	Proporsi prediksi yang benar
Precision	TP / (TP + FP)	Ketepatan prediksi kelas positif
Recall	TP / (TP + FN)	Proporsi positif aktual yang terdeteksi
F1-score	2 x Precision x Recall / (Precision + Recall)	Rata-rata harmonik precision dan recall

Akurasi hanya andal bila jumlah data tiap kelas relatif seimbang (Aldelemy & Abd-Alhameed, 2023), sehingga pada data tidak seimbang recall dan F1 kelas minoritas perlu diperhatikan.

2.6 Penelitian Terkait

Penelitian	Metode	Temuan
Hayder et al. (2023)	k-NN, Decision Tree, Naive Bayes, SVM; SMOTE dan seleksi fitur	Decision Tree terbaik dengan akurasi sekitar 91-92%; Naive Bayes sekitar 80-81%
Safarkhani & Moro (2021)	Decision Tree J48 dengan resampling dan seleksi fitur	Akurasi 94,39%
Aldelemy & Abd-Alhameed (2023)	Delapan klasifikator dengan stratified 5-fold CV pada 45.211 data	Decision Tree akurasi 0,891; Regresi Logistik 0,901; Gradient Boosting 0,906

BAB III
HASIL

3.1 Lingkungan Pengerjaan dan Dataset

Praktikum dikerjakan di Jupyter Notebook dengan bahasa Python menggunakan pustaka pandas, numpy, matplotlib, seaborn, dan scikit-learn. Dataset yang digunakan adalah dataset nomor 4 (Bank Customers) sesuai pembagian algoritma dan dataset, dengan algoritma Regresi Linear dan Decision Tree. Seluruh cell kode diberi watermark NIM 202431164 berupa komentar pada baris pertama cell, pesan cetak pada akhir output, dan teks NIM transparan pada setiap grafik.

Aspek	Hasil
Jumlah baris / kolom	45.211 / 17
Fitur numerik	age, balance, day, duration, campaign, pdays, previous (7)
Fitur kategorikal	job, marital, education, default, housing, loan, contact, month, poutcome (9)
Target	y (yes / no)
Missing value / duplikat	0 / 0
Nilai "unknown"	job 288; education 1.857; contact 13.020; poutcome 36.959
Kelas no / yes	39.922 (88,3%) / 5.289 (11,7%)

Fitur	Mean	Std	Min	Median	Max
age	40,9	10,6	18	39	95
balance	1.362,3	3.044,8	-8.019	448	102.127
day	15,8	8,3	1	16	31
duration	258,2	257,5	0	180	4.918
campaign	2,8	3,1	1	2	63
pdays	40,2	100,1	-1	-1	871
previous	0,6	2,3	0	0	275

3.1 Penjelasan Kode dan Output Setiap Cell

Berikut kode, penjelasan, dan output dari setiap cell pada file 202431164_UTS_PM.ipynb.

Cell 1: Import library dan fungsi watermark

```
[5]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
import os, json, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report,
                             r2_score, mean_absolute_error, mean_squared_error, f1_score)
warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
NIM = '202431164'
os.makedirs('fig', exist_ok=True)
RES = {}

def wm(fig):
    # watermark NIM pada setiap gambar
    fig.text(0.5, 0.5, NIM, fontsize=48, color='gray', alpha=0.15,
             ha='center', va='center', rotation=30)

def save(fig, name):
    wm(fig); fig.tight_layout(); fig.savefig(f'fig/{name}.png', dpi=130); plt.show()
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

```
[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Mengimpor seluruh pustaka (pandas, numpy, matplotlib, seaborn, scikit-learn), menyiapkan konstanta NIM, folder *fig*, dictionary RES untuk menyimpan hasil, serta fungsi *wm()* yang menempelkan watermark NIM pada gambar dan *save()* untuk menyimpan sekaligus menampilkan grafik.

Cell 2: Memuat dataset

[6]:
```
# ===== NIM 202431164 | Fajar Febriano Admosa =====
from pathlib import Path
FILE = '04_bank_customers.csv'

def find_csv(name):
    # cari file di folder kerja, folder induk, lalu Downloads/Desktop/Documents, lalu subfolder
    home = Path.home()
    for p in [Path.cwd()/name, Path.cwd().parent/name, home/'Downloads'/name,
              home/'Desktop'/name, home/'Documents'/name]:
        if p.exists(): return p
    for p in Path.cwd().rglob(name): return p
    return None

path = find_csv(FILE)
if path is None:   # jika tidak ketemu, minta pilih file lewat jendela dialog
    try:
        import tkinter as tk
        from tkinter import filedialog
        root = tk.Tk(); root.withdraw(); root.attributes('-topmost', True)
        sel = filedialog.askopenfilename(title='Pilih 04_bank_customers.csv', filetypes=[('CSV','*.csv')])
        root.destroy()
        path = Path(sel) if sel else None
    except Exception:
        path = None
if path is None:
    raise FileNotFoundError(f'{FILE} tidak ditemukan. Letakkan file di folder: {Path.cwd()}')

print('Dataset dibaca dari:', path)
df = pd.read_csv(path, sep=';')
print('Ukuran data (baris, kolom):', df.shape)
df.head()
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

Penjelasan: Mencari file 04_bank_customers.csv secara otomatis (folder kerja, folder induk, Downloads, Desktop, Documents, lalu subfolder); jika tidak ketemu, muncul jendela dialog untuk memilih file. Setelah itu data dibaca dengan pemisah titik koma (sep=";"), ukuran data dicetak (45.211 baris, 17 kolom), dan lima baris pertama ditampilkan.

Cell 3: Informasi umum dan kualitas data

[7]:
```
# ===== NIM 202431164 | Fajar Febriano Admosa =====
df.info()
print('\nMissing value per kolom:\n', df.isna().sum())
print('\nJumlah duplikat:', df.duplicated().sum())
print('\nNilai "unknown" per kolom kategorikal:')
print((df.select_dtypes('object') == 'unknown').sum()[lambda s: s > 0])
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

```
<class 'pandas.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   age         45211 non-null  int64
 1   job         45211 non-null  str
 2   marital     45211 non-null  str
 3   education   45211 non-null  str
 4   default     45211 non-null  str
 5   balance     45211 non-null  int64
 6   housing     45211 non-null  str
 7   loan        45211 non-null  str
 8   contact     45211 non-null  str
 9   day         45211 non-null  int64
 10  month       45211 non-null  str
 11  duration    45211 non-null  int64
 12  campaign    45211 non-null  int64
 13  pdays       45211 non-null  int64
 14  previous    45211 non-null  int64
 15  poutcome    45211 non-null  str
 16  y           45211 non-null  str
dtypes: int64(7), str(10)
memory usage: 5.9 MB

Missing value per kolom:
 age          0
job           0
marital       0
education     0
default       0
balance       0
housing       0
loan          0
contact       0
day           0
month         0
duration      0
campaign      0
pdays         0
previous      0
poutcome      0
y             0
dtype: int64

Jumlah duplikat: 0

Nilai "unknown" per kolom kategorikal:
job          288
education   1857
contact    13020
poutcome   36959
dtype: int64

[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Menampilkan tipe data dan jumlah non-null (info), jumlah missing value per kolom, jumlah data duplikat, serta jumlah nilai "unknown" pada kolom kategorikal. Hasil: tidak ada missing value dan duplikat; "unknown" ada di job, education, contact, poutcome.

Cell 4: Statistik deskriptif

```
# ===== NIM 202431164 | Fajar Febriano Admosa =====
df.describe().T
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

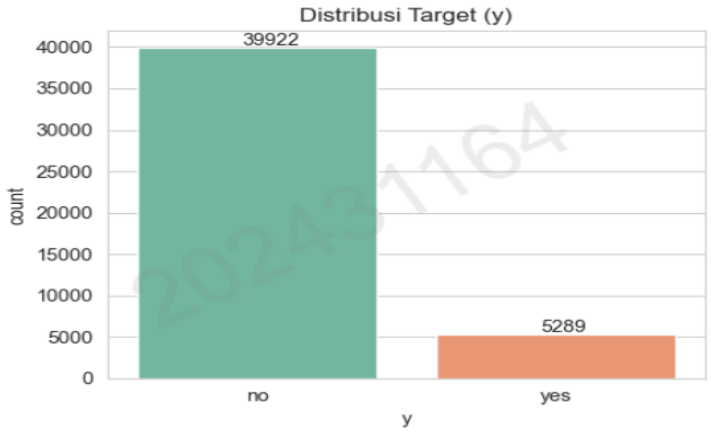
```
[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Menampilkan statistik deskriptif fitur numerik (count, mean, std, min, kuartil, max) untuk memahami skala dan sebaran data.

Cell 5: EDA - distribusi target

```
[9]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
vc = df['y'].value_counts()
print(vc, '\n', (vc/len(df)*100).round(2))
RES['target_counts'] = vc.to_dict()
fig, ax = plt.subplots(figsize=(5,4))
sns.countplot(x='y', data=df, ax=ax, palette='Set2')
for p in ax.patches: ax.annotate(int(p.get_height()), (p.get_x()+.35, p.get_height()+300))
ax.set_title('Distribusi Target (y)')
save(fig, 'eda_target')
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

```
y
no     39922
yes     5289
Name: count, dtype: int64
 y
no     88.3
yes    11.7
Name: count, dtype: float64
```

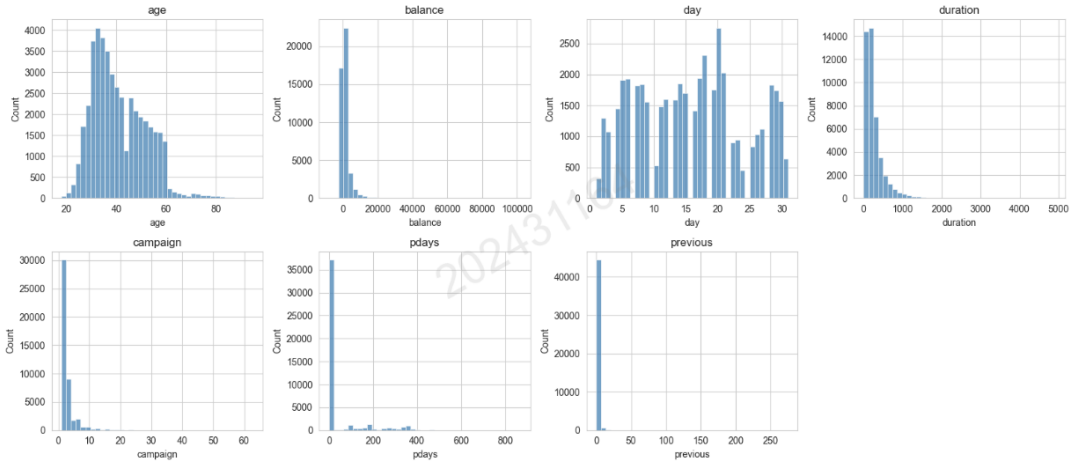



```
[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Menghitung jumlah dan persentase kelas target, menyimpannya ke RES, lalu membuat countplot dengan label jumlah. Menunjukkan ketidakseimbangan kelas (88,3% no, 11,7% yes).

Cell 6: EDA - distribusi numerik

```
[10]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
num_cols = df.select_dtypes(include='number').columns.tolist()
fig, axes = plt.subplots(2, 4, figsize=(16,7))
for ax, c in zip(axes.ravel(), num_cols):
    sns.histplot(df[c], bins=40, kde=False, ax=ax, color='steelblue'); ax.set_title(c)
axes.ravel()[-1].axis('off')
save(fig, 'eda_numeric')
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

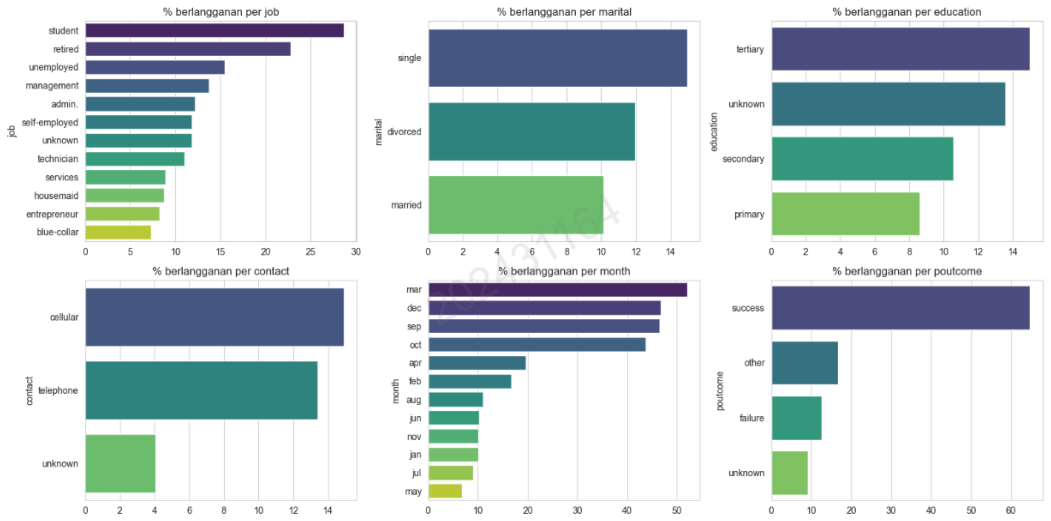



```
[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Memilih kolom numerik lalu membuat histogram tiap fitur dalam grid 2x4 untuk melihat bentuk distribusi dan kemencengan.

Cell 7: EDA - kategorikal terhadap target

```
[11]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
cat_cols = df.select_dtypes('object').columns.drop('y').tolist()
tmp = df.assign(y_bin=(df['y']=='yes').astype(int))
fig, axes = plt.subplots(2, 3, figsize=(16,8))
for ax, c in zip(axes.ravel(), ['job','marital','education','contact','month','poutcome']):
    r = tmp.groupby(c)['y_bin'].mean().sort_values(ascending=False)*100
    sns.barplot(x=r.values, y=r.index, ax=ax, palette='viridis'); ax.set_title(f'% berlangganan per {c}')
save(fig, 'eda_categorical')
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

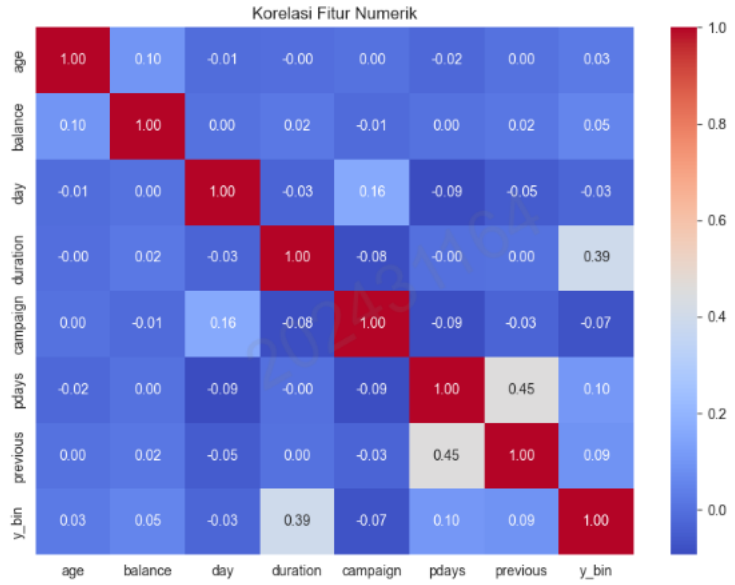



```
[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Mengubah target menjadi 0/1 sementara (y_bin), menghitung rata-rata (persentase berlangganan) per kategori untuk enam fitur kategorikal, dan menampilkannya dalam bar chart terurut.

Cell 8: EDA - korelasi

```
[12]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
corr = tmp[num_cols + ['y_bin']].corr()
fig, ax = plt.subplots(figsize=(8,6))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', ax=ax); ax.set_title('Korelasi Fitur Numerik')
save(fig, 'eda_corr')
print(corr['y_bin'].sort_values(ascending=False))
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```



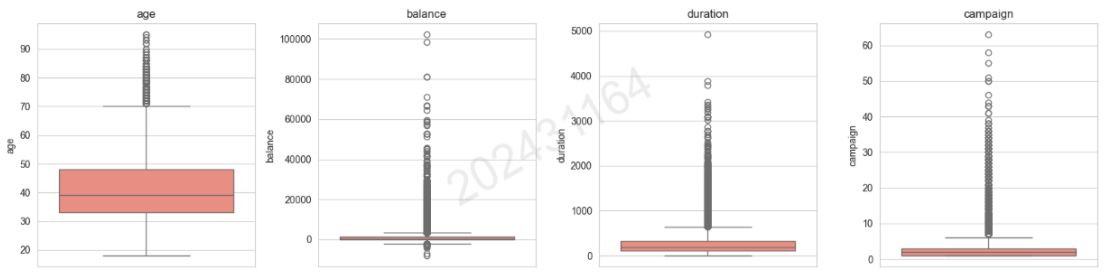

```
y_bin       1.000000
duration    0.394521
pdays       0.103621
previous    0.093236
balance     0.052838
age         0.025155
day        -0.028348
campaign   -0.073172
Name: y_bin, dtype: float64

[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Menghitung matriks korelasi fitur numerik dengan target, menampilkannya sebagai heatmap beranotasi, dan mencetak urutan korelasi terhadap target.

Cell 9: EDA - outlier

```
[13]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
fig, axes = plt.subplots(1, 4, figsize=(16,4))
for ax, c in zip(axes, ['age','balance','duration','campaign']):
    sns.boxplot(y=df[c], ax=ax, color='salmon'); ax.set_title(c)
save(fig, 'eda_outlier')
for c in ['age','balance','duration','campaign']:
    q1,q3 = df[c].quantile([.25,.75]); i = q3-q1
    n = ((df[c] < q1-1.5*i) | (df[c] > q3+1.5*i)).sum()
    print(f'{c:10s} outlier IQR: {n} ({n/len(df)*100:.1f}%)')
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```




```
age        outlier IQR: 487 (1.1%)
balance    outlier IQR: 4729 (10.5%)
duration   outlier IQR: 3235 (7.2%)
campaign   outlier IQR: 3064 (6.8%)

[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Membuat boxplot untuk age, balance, duration, campaign, serta menghitung jumlah outlier dengan aturan IQR (di luar Q1-1,5 IQR dan Q3+1,5 IQR).

Hasil: outlier IQR ditemukan pada balance (10,5%), duration (7,2%), campaign (6,8%), dan age (1,1%). Outlier dipertahankan karena merupakan nilai nyata dan Decision Tree tidak sensitif terhadap outlier.

Cell 10: Preprocessing data

```
[14]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
data = df.drop_duplicates().copy()
data['y'] = data['y'].map({'no':0, 'yes':1})
X = pd.get_dummies(data.drop(columns='y'), drop_first=True, dtype=int)
y = data['y']
print('Bentuk X setelah encoding:', X.shape)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print('Train:', X_train.shape, '| Test:', X_test.shape)
print('Proporsi y=1 train/test:', round(y_train.mean(),4), round(y_test.mean(),4))

sc = StandardScaler().fit(X_train[num_cols])
X_train_s, X_test_s = X_train.copy(), X_test.copy()
X_train_s[num_cols] = sc.transform(X_train[num_cols])
X_test_s[num_cols]  = sc.transform(X_test[num_cols])
RES['n_features'] = X.shape[1]; RES['n_train'] = len(X_train); RES['n_test'] = len(X_test)
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

```
Bentuk X setelah encoding: (45211, 42)
Train: (36168, 42) | Test: (9043, 42)
Proporsi y=1 train/test: 0.117 0.117

[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Preprocessing: menghapus duplikat, mengubah target yes/no menjadi 1/0, one-hot encoding fitur kategorikal (menjadi 42 fitur), membagi data 80:20 stratified (random_state=42), dan menstandardisasi fitur numerik dengan StandardScaler yang di-fit hanya pada data latih.

Cell 11: Regresi Linear - pelatihan dan evaluasi

```
[15]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
lr = LinearRegression().fit(X_train_s, y_train)
pred_cont = lr.predict(X_test_s)
pred_lr = (pred_cont >= 0.5).astype(int)

acc_lr = accuracy_score(y_test, pred_lr)
RES['lr'] = dict(accuracy=acc_lr, f1=f1_score(y_test, pred_lr), r2=r2_score(y_test, pred_cont),
                 mae=mean_absolute_error(y_test, pred_cont), mse=mean_squared_error(y_test, pred_cont),
                 rmse=float(np.sqrt(mean_squared_error(y_test, pred_cont))),
                 train_acc=accuracy_score(y_train, (lr.predict(X_train_s)>=0.5).astype(int)))
print({k: round(v,4) for k,v in RES['lr'].items()})
print('\nClassification report:\n', classification_report(y_test, pred_lr))
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

```
{'accuracy': 0.8984, 'f1': 0.3902, 'r2': 0.2978, 'mae': 0.1615, 'mse': 0.0725, 'rmse': 0.2693, 'train_acc': 0.8998}

Classification report:
              precision    recall  f1-score   support

           0       0.91      0.98      0.94      7985
           1       0.65      0.28      0.39      1058

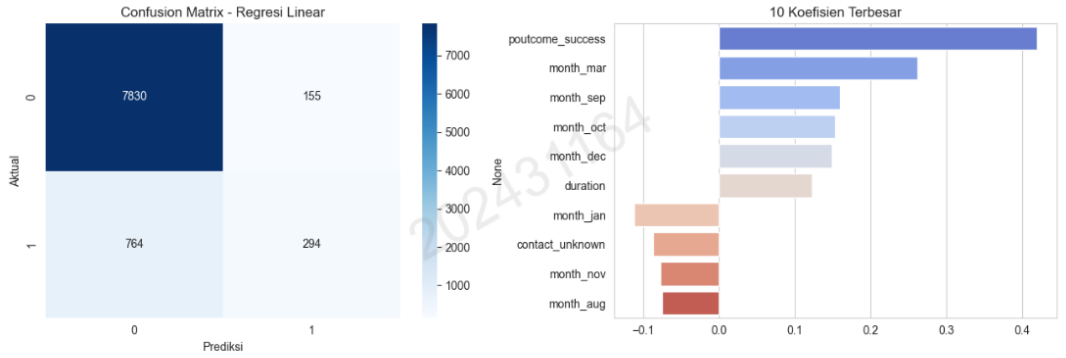
    accuracy                           0.90      9043
   macro avg       0.78      0.63      0.67      9043
weighted avg       0.88      0.90      0.88      9043


[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Melatih LinearRegression pada data terstandardisasi, memprediksi data uji, mengubah skor kontinu menjadi kelas dengan threshold 0,5, lalu menghitung akurasi, F1, R2, MAE, MSE, RMSE, akurasi latih, dan classification report.

Cell 12: Regresi Linear - visualisasi

```
[16]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
cm = confusion_matrix(y_test, pred_lr); RES['lr']['cm'] = cm.tolist()
fig, axes = plt.subplots(1, 2, figsize=(13,4.5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0]); axes[0].set_title('Confusion Matrix - Regresi Linear')
axes[0].set_xlabel('Prediksi'); axes[0].set_ylabel('Aktual')
coef = pd.Series(lr.coef_, index=X.columns).sort_values(key=abs, ascending=False).head(10)
sns.barplot(x=coef.values, y=coef.index, ax=axes[1], palette='coolwarm'); axes[1].set_title('10 Koefisien Terbesar')
save(fig, 'lr_result')
RES['lr']['top_coef'] = coef.round(4).to_dict()
print(coef)
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```




```
poutcome_success    0.419527
month_mar           0.262161
month_sep           0.159260
month_oct           0.153448
month_dec           0.148903
duration            0.122514
month_jan          -0.112363
contact_unknown    -0.087313
month_nov          -0.077500
month_aug          -0.075220
dtype: float64

[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Membuat confusion matrix dan grafik 10 koefisien terbesar (berdasarkan nilai absolut) untuk menginterpretasikan pengaruh tiap fitur pada Regresi Linear.

Cell 13: Decision Tree - baseline

```
[17]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
dt0 = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
RES['dt_base'] = dict(train_acc=dt0.score(X_train, y_train), accuracy=dt0.score(X_test, y_test),
                      f1=f1_score(y_test, dt0.predict(X_test)), depth=int(dt0.get_depth()))
print('Decision Tree tanpa batasan:', {k: round(v,4) for k,v in RES['dt_base'].items()})
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

```
Decision Tree tanpa batasan: {'train_acc': 1.0, 'accuracy': 0.8776, 'f1': 0.4882, 'depth': 35}

[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Melatih Decision Tree baseline tanpa batas kedalaman, lalu mengukur akurasi latih, akurasi uji, F1, dan kedalaman pohon. Digunakan untuk menunjukkan overfitting (latih 100%).

Cell 14: Decision Tree - tuning GridSearchCV

```
[19]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
    {'max_depth':[3,5,7,10,15], 'min_samples_leaf':[1,10,30,50], 'criterion':['gini','entropy']},
    cv=5, scoring='accuracy', n_jobs=-1).fit(X_train, y_train)
dt = grid.best_estimator_
pred_dt = dt.predict(X_test)
print('Parameter terbaik:', grid.best_params_, '| CV acc:', round(grid.best_score_,4))
RES['dt'] = dict(accuracy=accuracy_score(y_test, pred_dt), f1=f1_score(y_test, pred_dt),
                 train_acc=dt.score(X_train, y_train), cv_acc=grid.best_score_,
                 params=grid.best_params_, depth=int(dt.get_depth()))
print({k: (round(v,4) if isinstance(v,float) else v) for k,v in RES['dt'].items()})
print('\nClassification report:\n', classification_report(y_test, pred_dt))
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

```
Parameter terbaik: {'criterion': 'gini', 'max_depth': 5, 'min_samples_leaf': 30} | CV acc: 0.9033
{'accuracy': 0.899, 'f1': 0.4218, 'train_acc': 0.9045, 'cv_acc': np.float64(0.9033), 'params': {'criterion': 'gini', 'max_depth': 5, 'min_samples_lea
f': 30}, 'depth': 5}

Classification report:
              precision    recall  f1-score   support

           0       0.91      0.98      0.94      7985
           1       0.64      0.31      0.42      1058

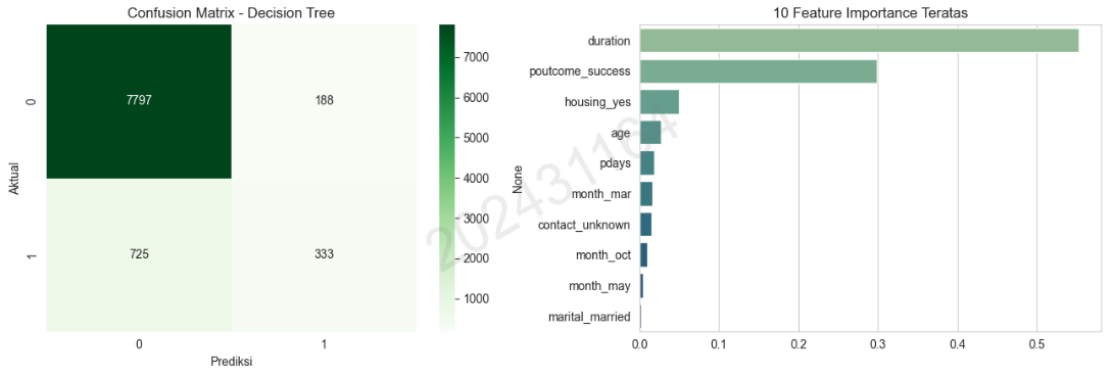
    accuracy                           0.90      9043
   macro avg       0.78      0.65      0.68      9043
weighted avg       0.88      0.90      0.88      9043


[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Melakukan GridSearchCV 5-fold pada max_depth, min_samples_leaf, dan criterion untuk mencari konfigurasi terbaik, melatih ulang model terbaik, lalu mencetak parameter, akurasi CV, akurasi uji, F1, dan classification report.

Cell 15: Decision Tree - visualisasi hasil

```
[20]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
cm2 = confusion_matrix(y_test, pred_dt); RES['dt']['cm'] = cm2.tolist()
fig, axes = plt.subplots(1, 2, figsize=(13,4.5))
sns.heatmap(cm2, annot=True, fmt='d', cmap='Greens', ax=axes[0]); axes[0].set_title('Confusion Matrix - Decision Tree')
axes[0].set_xlabel('Prediksi'); axes[0].set_ylabel('Aktual')
imp = pd.Series(dt.feature_importances_, index=X.columns).sort_values(ascending=False).head(10)
sns.barplot(x=imp.values, y=imp.index, ax=axes[1], palette='crest'); axes[1].set_title('10 Feature Importance Teratas')
save(fig, 'dt_result')
RES['dt']['top_imp'] = imp.round(4).to_dict()
print(imp)
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```



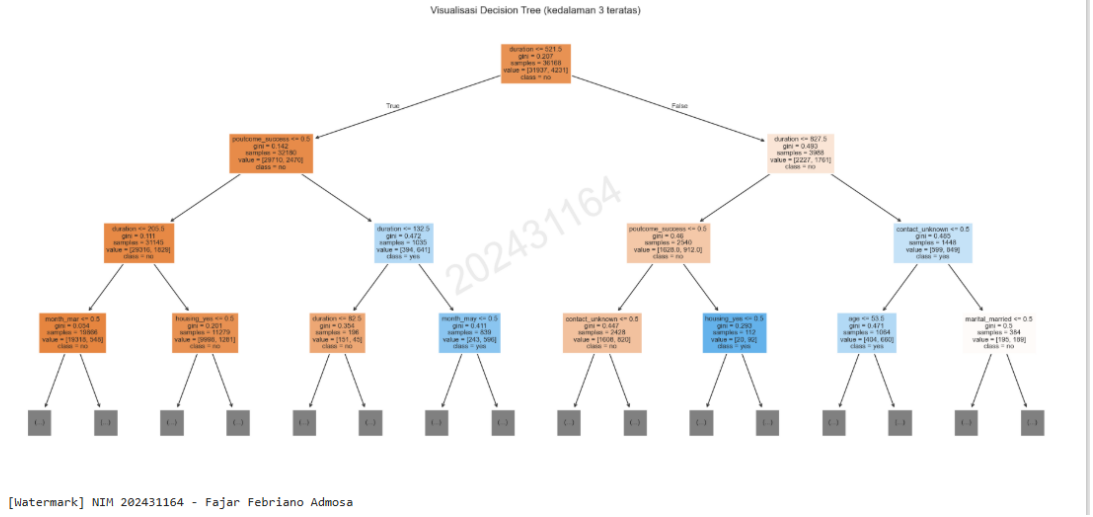

```
duration           0.553283
poutcome_success   0.298430
housing_yes        0.050157
age                0.026984
pdays              0.018788
month_mar          0.016774
contact_unknown    0.015656
month_oct          0.010212
month_may          0.005044
marital_married    0.002169
dtype: float64

[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Menampilkan confusion matrix dan 10 feature importance teratas dari Decision Tree terbaik.

Cell 16: Decision Tree - visualisasi pohon

```
[21]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
fig = plt.figure(figsize=(18,8))
plot_tree(dt, max_depth=3, feature_names=X.columns, class_names=['no','yes'], filled=True, fontsize=8)
plt.title('Visualisasi Decision Tree (kedalaman 3 teratas)')
save(fig, 'dt_tree')
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```



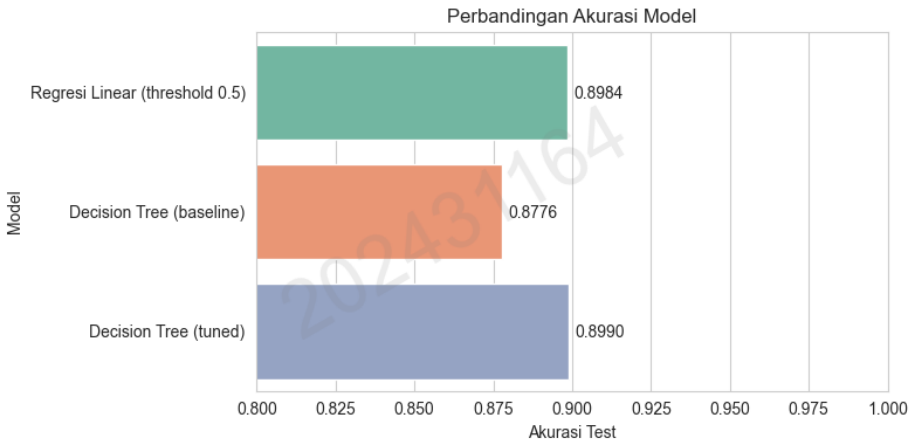

```
[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Memvisualisasikan pohon keputusan hingga kedalaman 3 dengan plot_tree, lengkap dengan nama fitur dan kelas, agar aturan pemisahan dapat dibaca.

Cell 17: Perbandingan model dan kesimpulan

```
[22]: # ===== NIM 202431164 | Fajar Febriano Admosa =====
comp = pd.DataFrame({
  'Model': ['Regresi Linear (threshold 0.5)', 'Decision Tree (baseline)', 'Decision Tree (tuned)'],
  'Akurasi Test': [RES['lr']['accuracy'], RES['dt_base']['accuracy'], RES['dt']['accuracy']],
  'F1 (kelas yes)': [RES['lr']['f1'], RES['dt_base']['f1'], RES['dt']['f1']],
}).round(4)
print(comp.to_string(index=False))
fig, ax = plt.subplots(figsize=(8,4))
sns.barplot(data=comp, x='Akurasi Test', y='Model', ax=ax, palette='Set2'); ax.set_xlim(0.8, 1.0)
for p in ax.patches: ax.annotate(f'{p.get_width():.4f}', (p.get_width()+.002, p.get_y()+.45))
ax.set_title('Perbandingan Akurasi Model')
save(fig, 'comparison')
best = comp.sort_values('Akurasi Test').iloc[-1]
print('\nModel terbaik:', best['Model'], '| akurasi =', best['Akurasi Test'])
json.dump(RES, open('results.json','w'), indent=2, default=float)
print('\n[Watermark] NIM 202431164 - Fajar Febriano Admosa')
```

```
                      Model  Akurasi Test  F1 (kelas yes)
Regresi Linear (threshold 0.5)        0.8984          0.3902
      Decision Tree (baseline)        0.8776          0.4882
         Decision Tree (tuned)        0.8990          0.4218
```




```
Model terbaik: Decision Tree (tuned) | akurasi = 0.899

[Watermark] NIM 202431164 - Fajar Febriano Admosa
```

Penjelasan: Menyusun tabel perbandingan akurasi dan F1 ketiga model, menggambar bar chart akurasi, menentukan model terbaik, dan menyimpan seluruh hasil ke results.json.

3.2 Hasil Evaluasi Model

Regresi Linear

Model Regresi Linear dilatih pada 36.168 data latih dengan 42 fitur hasil encoding dan diuji pada 9.043 data uji. Setelah thresholding 0,5, akurasi data uji sebesar **89,84%** (akurasi latih 89,98%). Selisih yang kecil antara akurasi latih dan uji menunjukkan model tidak overfitting

Metrik regresi	Nilai
R2	0,2978
MAE	0,1615
MSE	0,0725
RMSE	0,2693

Nilai R2 sebesar 0,30 berarti model hanya menjelaskan sekitar 30% variasi target. Hal ini wajar karena target biner kurang cocok untuk model linear (lihat bagian 2.3). Koefisien terbesar: poutcome_success (0,42), month_mar (0,26), month_sep (0,16), month_oct (0,15), month_dec (0,15), duration (0,12). Fitur poutcome_success dan bulan tertentu menaikkan peluang berlangganan, sejalan dengan temuan EDA.

Decision Tree

Decision Tree tanpa batasan (kedalaman 35) mencapai akurasi latih 100% tetapi akurasi uji hanya 87,76%, tanda overfitting. Setelah *GridSearchCV* 5-fold, parameter terbaik adalah criterion = gini, max_depth = 5, dan min_samples_leaf = 30, dengan akurasi cross-validation 90,33%. Pada data uji model ini mencapai akurasi **89,90%** (akurasi latih 90,45%), sehingga lebih general dibanding pohon penuh.

Fitur paling berpengaruh menurut feature importance: duration (55,3%), poutcome_success (29,8%), housing_yes (5,0%), age (2,7%), pdays (1,9%). Dua fitur teratas (duration dan poutcome_success) menyumbang sekitar 85% dari total importance.

Perbandingan Model

Model	Akurasi uji	Precision (yes)	Recall (yes)	F1 (yes)
Regresi Linear	89,84%	0,6548	0,2779	0,3902
Decision Tree (baseline)	87,76%	-	-	0,4882
Decision Tree (tuned)	89,90%	0,6392	0,3147	0,4218

Model	Kelas	Precision	Recall	F1-score
Regresi Linear	no	0,9111	0,9806	0,9446
	yes	0,6548	0,2779	0,3902
Decision Tree (tuned)	no	0,9149	0,9765	0,9447
	yes	0,6392	0,3147	0,4218

Model	TN	FP	FN	TP
Regresi Linear	7.830	155	764	294
Decision Tree (tuned)	7.797	188	725	333

3.1 Analisis dan Pembahasan

1. Akurasi. Decision Tree hasil tuning memperoleh akurasi tertinggi (89,90%), selisih sangat tipis dari Regresi Linear (89,84%). Decision Tree juga memiliki recall kelas "yes" yang lebih baik: 333 dari 1.058 nasabah "yes" berhasil terdeteksi, dibanding 294 pada Regresi Linear. Decision Tree dapat menangkap hubungan non-linear dan interaksi antarfitur, misalnya

kombinasi durasi panggilan yang panjang dengan hasil kampanye sebelumnya yang sukses, yang tidak dapat ditangkap model linear.

2. Ketidakseimbangan kelas. Kelas "no" mencakup 88,3% data, sehingga tebakan "selalu no" sudah memberi akurasi sekitar 88,3%. Kedua model hanya sedikit di atas angka itu, dan recall kelas "yes" masih rendah (27,8% dan 31,5%), artinya banyak calon nasabah terlewat. Kondisi ini sesuai dengan temuan Hayder et al. (2023) bahwa akurasi kurang efektif pada data tidak seimbang. Peningkatan dapat dicoba dengan class_weight, penyesuaian threshold, atau resampling seperti SMOTE.

3. Pembatasan kedalaman pohon. Pohon tanpa batas menghafal data latih (100%) tetapi menurun pada data uji. Pembatasan max_depth dan min_samples_leaf terbukti meningkatkan generalisasi, sesuai teori bahwa overfitting pada Decision Tree perlu dikendalikan lewat hiperparameter (Aldelemy & Abd-Alhameed, 2023).

4. Fitur duration. Duration dominan pada kedua model. Durasi panggilan baru diketahui setelah panggilan selesai, sehingga untuk penerapan prediktif nyata fitur ini berpotensi menyebabkan data leakage. Hasil di sini dipakai untuk keperluan analisis praktikum.

5. Perbandingan dengan literatur. Akurasi Decision Tree pada penelitian ini (89,90%) sejalan dengan Aldelemy dan Abd-Alhameed (2023) yang melaporkan akurasi 0,891 untuk Decision Tree pada dataset berukuran sama dengan validasi silang 5-fold. Hayder et al. (2023) dan Safarkhani dan Moro (2021) memperoleh akurasi lebih tinggi (91-94%), tetapi memakai penyeimbangan data, seleksi fitur, atau versi dataset yang berbeda, sehingga angkanya tidak dapat dibandingkan langsung.

BAB IV
PENUTUP

4.1 Kesimpulan

1. Dataset Bank Customers berisi 45.211 baris dan 17 kolom, bersih dari missing value dan duplikat, tetapi tidak seimbang (88,3% "no" dan 11,7% "yes"). Preprocessing yang dilakukan meliputi encoding target, one-hot encoding fitur kategorikal, pembagian data 80:20 terstratifikasi, dan standardisasi fitur numerik.
2. Regresi Linear dengan threshold 0,5 memperoleh akurasi 89,84% (R2 = 0,2978), sedangkan Decision Tree tanpa batasan hanya 87,76% karena overfitting.
3. Decision Tree hasil tuning (criterion gini, max_depth 5, min_samples_leaf 30) memberikan hasil terbaik dengan akurasi 89,90% dan F1 kelas "yes" 0,4218, sedikit lebih baik dari Regresi Linear.
4. Fitur yang paling menentukan adalah durasi panggilan (duration) dan hasil kampanye sebelumnya (poutcome_success). Selisih akurasi kedua model kecil dan keduanya hanya sedikit di atas akurasi tebakan kelas mayoritas, sehingga recall kelas "yes" menjadi hal yang masih perlu ditingkatkan.

4.2 Saran

1. Menangani ketidakseimbangan kelas dengan class_weight, SMOTE, atau penyesuaian threshold untuk meningkatkan recall kelas "yes".
2. Menggunakan validasi silang dan menguji tanpa fitur duration agar model lebih realistis untuk prediksi sebelum panggilan dilakukan.
3. Membandingkan dengan algoritma lain seperti regresi logistik, Random Forest, atau Gradient Boosting.

DAFTAR PUSTAKA

Aldelemy, A., & Abd-Alhameed, R. A. (2023). Binary classification of customer's online purchasing behavior using machine learning. Journal of Techniques, 5(2), 163-186. https://doi.org/10.51173/jt.v5i2.1226

Hayder, I. M., Al Ali, G. A. N., & Younis, H. A. (2023). Predicting reaction based on customer's transaction using machine learning approaches. International Journal of Electrical and Computer Engineering (IJECE), 13(1), 1086-1096. https://doi.org/10.11591/ijece.v13i1.pp1086-1096

Safarkhani, F., & Moro, S. (2021). Improving the accuracy of predicting bank depositor's behavior using a decision tree. Applied Sciences, 11(19), 9016. https://doi.org/10.3390/app11199016